

# AI Operations (AIOps)

## Module 3 Assignment — Infrastructure & Containerization

*Student Edition*

| Total Marks | Weight             | Submission                                                                                 |
|-------------|--------------------|--------------------------------------------------------------------------------------------|
| 40          | 8% of course grade | Git repository link (Dockerfiles + Compose file + YAML + evidence) + a 2-page PDF write-up |

### Before You Start

This assignment asks you to APPLY what the Lectures on Dockers and Kubernetes taught, to a scenario you have not seen before. The Dockerfiles, docker-compose.yml, and Kubernetes manifests you submit should be written by you, for this application — copying the structure of the lecture's hyperparameter-search or predictor examples without adapting it to this scenario's actual requirements will not satisfy the rubric criteria below.

**Everything in this assignment is CPU-only. No GPU is required or expected anywhere.**

- Questions 1–2 are about Docker: a naive vs. multi-stage build, and multi-container orchestration with Docker Compose (a tool not covered hands-on in lecture — you are expected to read its documentation and adapt what you know about Dockerfiles to it).
- Questions 3–4 are about Kubernetes: an Indexed Job for a new batch workload, and a Deployment exercised for its self-healing and rolling-update behavior (not autoscaling, which this assignment does not require).
- Each question below includes its own evaluation rubric, so you know exactly what evidence you need to produce before you submit.

## The Scenario: A Spam-Detection API

All four questions build around ONE small application: a REST API that classifies a short text message as “spam” or “ham” (not spam), using a TF-IDF vectorizer and a Naive Bayes classifier — deliberately simple, so the ML modeling itself is not the challenge; your Docker and Kubernetes skills are.

### Generate Your Dataset

Run the following script yourself to generate a deterministic, synthetic dataset of 1,000 labeled messages (no external download needed):

```
import random
random.seed(42)

SPAM_TEMPLATES = [
    "WIN a FREE {prize} now! Click here: {url}",
    "Congratulations! You have WON a {prize}. Claim NOW at {url}",
    "URGENT: Your account will be suspended. Verify at {url}",
    "Limited time offer! Get {prize} FREE, click {url} today",
    "You've been selected for a {prize}! Reply YES to claim",
    "Cash prize alert: claim your {prize} before it expires! {url}",
]
HAM_TEMPLATES = [
    "Hey, are we still meeting for {activity} on {day}?",
    "Can you send me the notes from {activity} class?",
    "Don't forget about {activity} this {day}, see you there",
    "Thanks for helping with {activity} yesterday",
    "Running a bit late for {activity}, be there in 10 min",
    "What time does {activity} start on {day}?",
]
PRIZES = ["iPhone", "cash prize", "gift card", "vacation", "laptop"]
URLS = ["bit.ly/xyz123", "tinyurl.com/abc", "win-now.co/claim"]
ACTIVITIES = ["lunch", "the study group", "basketball", "the project meeting"]
DAYS = ["Monday", "Friday", "tomorrow", "the weekend"]

rows = []
for i in range(1000):
    if random.random() < 0.3:
        t = random.choice(SPAM_TEMPLATES)
        msg = t.format(prize=random.choice(PRIZES), url=random.choice(URLS))
        rows.append((msg, "spam"))
    else:
        t = random.choice(HAM_TEMPLATES)
        msg = t.format(activity=random.choice(ACTIVITIES), day=random.choice(DAYS))
        rows.append((msg, "ham"))

import pandas as pd
pd.DataFrame(rows, columns=["text", "label"]).to_csv("spam_dataset.csv", index=False)
```

## The Required API Contract

Whatever framework you choose (FastAPI is recommended), your service must expose:

- POST /predict — accepts JSON {"text": "..."}, returns JSON {"label": "spam"} or {"label": "ham"}.
- GET /healthz — returns HTTP 200 once the model is loaded (used by Kubernetes probes in Question 4).

Train a TfidfVectorizer + MultinomialNB pipeline on spam\_dataset.csv, save it with joblib, and load it at application startup — this part is a direct, expected application of patterns from Module 1 and Kubernetes Lecture's predictor app, not something you need to redesign from scratch.

### Question 1 — Single-Stage vs. Multi-Stage Docker (10 marks)

Package your spam-detection API as a Docker image.

1. Write a naive, single-stage Dockerfile that successfully builds and runs your API. Report the resulting image size (docker images).
2. Rewrite it as a multi-stage build. Report the new image size and the percentage reduction.
3. In 2-3 sentences, explain SPECIFICALLY why the multi-stage image is smaller — name what was left behind in the builder stage for YOUR image (not a generic answer).

#### Evaluation Rubric

| Criterion                                                                                                                                                                   | Marks |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Naive Dockerfile builds and runs successfully; container correctly serves /predict and /healthz; size reported                                                              | 3     |
| Multi-stage Dockerfile builds and runs successfully with identical behavior; size + % reduction reported                                                                    | 4     |
| Explanation is specific to this image's actual contents (named base image difference, excluded build tools/caches), not a generic textbook definition of multi-stage builds | 3     |

## Question 2 — Multi-Container Orchestration with Docker Compose (10 marks)

Extend your spam-detection API with a caching layer, and run the two-container application with Docker Compose. The caching layer should be implemented using Redis, which is easier to learn by yourself.

1. Modify your API so that before running a prediction, it checks a Redis cache for that exact input text; on a cache MISS, it computes the prediction, stores it in Redis (with a TTL of your choice), and returns it; on a cache HIT, it returns the cached label without recomputing.
2. Write a docker-compose.yml defining two services: api (built from your multi-stage Dockerfile) and cache (the official redis:7-alpine image), connected so your API can reach Redis using the service name as its hostname.
3. Bring the stack up with docker compose up and provide evidence — logs, timing output, or a short script — showing that a repeated identical request is measurably faster on the second call (cache hit) than the first (cache miss).
4. In 2–3 sentences: what specific problem does Docker Compose solve that a single Dockerfile/container cannot, and how does its scope differ from what Kubernetes does?

### Evaluation Rubric

| Criterion                                                                                                                                                 | Marks |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| API correctly checks/writes the cache and still returns correct predictions on both hits and misses                                                       | 4     |
| docker-compose.yml correctly defines both services with working inter-service networking (service-name-as-hostname)                                       | 3     |
| Concrete evidence of a measurable cache-hit speedup (not just an assertion that caching “should” help)                                                    | 2     |
| Compose-vs-Kubernetes explanation correctly identifies Compose as single-host multi-container orchestration vs. Kubernetes as cluster-scale orchestration | 1     |

### Question 3 — Kubernetes Indexed Job: Parallel Data Validation (10 marks)

A separate (unrelated to spam detection) batch workload: you are given 8 partitioned CSV shards of user signup records, each containing some rows with malformed email addresses or missing required fields. Design a Kubernetes Indexed Job where each pod validates exactly ONE shard and reports how many invalid rows it found. (You may generate 8 small synthetic shard files yourself, following the same deterministic-generation pattern used for the dataset in this assignment's scenario section — or adapt the Kubernetes Lecture dataset-generation approach to produce shards with a known, seeded number of deliberately invalid rows per shard.)

1. Assume your available cluster has exactly 2 nodes with 2 allocatable CPUs each (4 total). Choose and justify a parallelism value for completions: 8, then write the full Indexed Job manifest: your chosen parallelism, CPU requests/limits consistent with your justification, and the Downward API fields needed to know which pod ran on which node.
2. Apply your Job and provide `kubectl get pods -o wide` evidence that your chosen parallelism value was actually reached (i.e., that many pods really did run concurrently, not just that the manifest permits it).
3. Using the Kubernetes API to read pod logs (not a shared volume — briefly justify why not, referencing what you learned about minikube's storage provisioner), collect and report each shard's invalid-row count.
4. If your cluster instead had 3 nodes with 2 CPUs each (6 total) available, would you change your parallelism choice? Justify your answer in 2–3 sentences.

#### Evaluation Rubric

| Criterion                                                                                                                        | Marks |
|----------------------------------------------------------------------------------------------------------------------------------|-------|
| Parallelism choice is justified with an actual calculation/argument tied to the stated 4-CPU constraint, not an arbitrary number | 3     |
| Valid Indexed Job manifest: completions: 8, correct restartPolicy, CPU requests = limits, both Downward API fields present       | 2     |
| Real <code>kubectl get pods -o wide</code> evidence showing the chosen parallelism was reached                                   | 2     |
| Results correctly collected via the Kubernetes API with a working justification for avoiding shared-volume storage               | 2     |
| 6-CPU re-justification is grounded in a real resource argument, not just restated from part 1                                    | 1     |

## Question 4 — Kubernetes Deployments: Self-Healing & Rolling Updates (10 marks)

Deploy your spam-detection API (Question 1) as a Kubernetes Deployment with 2 replicas and a Service — no autoscaler is required or expected for this question.

1. Write a Deployment manifest (2 replicas, CPU requests/limits, a readinessProbe against /healthz) and a Service manifest exposing it.
2. Self-healing: manually delete one running pod (`kubectl delete pod <name>`) and provide `kubectl get pods` evidence that Kubernetes recreated it automatically. In 2–3 sentences, explain WHICH controller is responsible for this and how it decides a replacement is needed.
3. Rolling update: make a small, visible change to your API (e.g., add a version string to the /healthz response), build it as a new image tag, update the Deployment to use it, and use `kubectl rollout status` and `kubectl rollout history` to demonstrate the update completed without downtime. In 2–3 sentences, explain what would have happened differently if this application had been deployed as a Job instead of a Deployment.
4. In 2–3 sentences: contrast a Deployment's self-healing/rolling-update behavior with a Job's run-to-completion behavior from Question 3 — why is each the right fit for its respective workload (serving vs. batch)?

### Evaluation Rubric

| Criterion                                                                                                                                                  | Marks |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Valid Deployment (2 replicas, resources, readinessProbe) and Service manifests, both applied successfully                                                  | 3     |
| Real evidence of self-healing (pod recreated after manual deletion) with a correct explanation naming the ReplicaSet/Deployment controller mechanism       | 3     |
| Real evidence of a completed rolling update (rollout status/history output) with a correct Job-vs-Deployment contrast for this specific scenario           | 3     |
| Final Deployment-vs-Job discussion correctly ties each object's behavior to its intended workload shape (long-running/serving vs. run-to-completion/batch) | 1     |

### Overall Rubric Summary

| Question | Marks | Skill Assessed                                                                                    |
|----------|-------|---------------------------------------------------------------------------------------------------|
| Q1       | 10    | Multi-stage Docker builds and image size optimization, applied to a new application               |
| Q2       | 10    | Multi-container orchestration with Docker Compose (self-directed application of new material)     |
| Q3       | 10    | Kubernetes Indexed Jobs: design justification and empirical verification for a new batch workload |
| Q4       | 10    | Kubernetes Deployments: self-healing and rolling updates                                          |
| Total    | 40    |                                                                                                   |